

Table of Contents

Week 10 — Abstraction: Abstract Classes

The Fourth Pillar (Part 1): Defining “What” Without Deciding “How”

Lesson Overview

Hour Plan

Recap of Week 9 (4 min)

Part 1 — Why Some Parents Should Never Be Built Directly (11 min)

Open with a story (3 min)

Definition

Three Real-World “Category, Not a Real Thing” Scenarios

What Abstraction Is NOT

Part 2 — Abstract Methods: A Contract With No Body (11 min)

Open with a story (2 min)

Definition

Abstract Method vs Regular Method — Side-by-Side

Why Force It? The Safety Net

Part 3 — Abstract Class vs Regular Parent Class (10 min)

Open with a question (2 min)

Side-by-Side Comparison

The Decision Rule

Part 4 — Mixing Abstract and Concrete: Partial Blueprints (9 min)

Open with a story (2 min)

Abstract Classes Can Have BOTH Kinds of Methods

Part 5 — Java as Illustration (8 min)

Reading Abstraction as a Concept

What Happens If You Try to Skip It

Polymorphism Still Works Exactly the Same Way

Reading as Plain English

Test Checkpoint

Common Misconceptions

Extension Challenges

Basic — Should It Be Abstract?

Intermediate — Regular or Abstract Method?

● Advanced — Design a Mixed Abstract Class

Lesson Summary

Homework (Prep for Interfaces, Week 11)

Week 10 — Abstraction: Abstract Classes

The Fourth Pillar (Part 1): Defining “What” Without Deciding “How”

Date: Wednesday, 5 August 2026 | **Time:** 6:30 PM – 7:30 PM | **Course:** Object-Oriented Programming Concepts

Lesson Overview

Item	Detail
Topic	Abstraction — abstract classes and abstract methods that force children to provide their own implementation
Date & Time	Wednesday, 5 August 2026 · 6:30 PM – 7:30 PM
Duration	1 hour
Format	Online (Google Meet) — Concept explanation + real-world discussion + light illustration
Prerequisites	Week 6 (Inheritance), Week 7 (Overriding), Week 9 (Polymorphism — especially “a parent action with no sensible default”)
Focus	OOP concept understanding — Java used only as illustration
Outcome	Students can explain why some parent classes should never be built directly, define “abstract method” as an action with no body that forces children to implement it, distinguish an abstract class from a regular parent class, and explain why abstraction and polymorphism work together

Hour Plan

00:00 – 00:04	Recap of Week 9 + Homework check
00:04 – 00:15	Part 1 – Why Some Parents Should Never Be Built Directly
00:15 – 00:26	Part 2 – Abstract Methods: A Contract With No Body
00:26 – 00:36	Part 3 – Abstract Class vs Regular Parent Class
00:36 – 00:45	Part 4 – Mixing Abstract and Concrete: Partial Blueprints

 **Google Meet tip:** Keep *Week10_Summary_Examples.md* open in a second tab — it has a completely different set of real-world stories (musical instruments and employee contracts) ready for every part of this lesson.

Recap of Week 9 (4 min)

Ask students verbally:

1. "What does polymorphism mean, in one sentence?"
→ *Calling the same action on different objects produces behavior specific to each object's actual type.*
2. "From homework — your `Shape.area()` action had no real formula, it just returned 0. Why couldn't `Shape` have one correct formula that worked for every shape?"
→ *Every shape calculates area completely differently — there is no single formula that makes sense for a generic, undefined shape.*
3. "If `area()` on a generic `Shape` doesn't mean anything real, should anyone ever be allowed to create a plain `Shape` object and call `area()` on it?"
→ *No — a "shape" that isn't a circle, rectangle, or triangle doesn't really exist. It's just a category, not a real thing.*
4. Transition question: "What if we could tell Java: 'No one is allowed to create a plain Shape object — Shape only exists so that Circle, Rectangle, and Triangle can build on it, and each of THEM must provide their own `area()`'? That rule is today's entire lesson."

Part 1 — Why Some Parents Should Never Be Built Directly (11 min)

Open with a story (3 min)

"Think about the word 'Vehicle.' Have you ever seen an actual, physical object that is just a 'vehicle' and nothing more specific? Not a car, not a motorbike, not a bicycle — just a bare 'vehicle,' sitting in a parking lot?

No — because 'vehicle' is a category, not a real thing you can build. Every actual vehicle you have ever seen was ALWAYS a more specific type: a car, a motorbike, a bicycle. 'Vehicle' only exists as an idea that ties them together — it describes what they have in common, but it was never meant to be built on its own."

Definition

Abstraction (through abstract classes) is the OOP concept of defining a parent class that describes what its children have in common — without ever intending for that parent to be built as a real, standalone object. An **abstract class** is a parent class that cannot be directly instantiated; it exists only to be extended.

Three Real-World “Category, Not a Real Thing” Scenarios

Work through each one. Ask: “Have you ever seen this thing exist on its own, with no more specific type? Or does it only ever appear as a specific child?”

SCENARIO 1 – SHAPES (from Week 9 homework)

Category: Shape
Always appears as: Circle, Rectangle, Triangle – never as a bare "Shape" with no defined form

Why abstract?

"Shape" only describes what circles, rectangles, and triangles have in common (they all have an area). It was never meant to be a real object on its own.

SCENARIO 2 – BANK ACCOUNTS

Category: BankAccount
Always appears as: SavingsAccount, CheckingAccount – never as a bare "BankAccount" with no rules attached

Why abstract?

Every real bank account has specific interest rules, withdrawal limits, and fees. A "BankAccount" with none of these defined isn't a real account a bank would ever open.

SCENARIO 3 – EMPLOYEES (payroll system)

Category: Employee
Always appears as: FullTimeEmployee, ContractEmployee – never as a bare "Employee" with no pay structure

Why abstract?

Every real employee has a specific way their pay is calculated. A generic "Employee" with no pay formula

isn't someone payroll could actually pay.

What Abstraction Is NOT

NOT ABSTRACTION:

A parent class that is perfectly fine to use on its own
→ "Person" (Week 6) makes sense as a real, standalone object – a generic person can exist and introduce() themselves without being a Student or Teacher

Simply not creating an object of a class, by choice
→ choosing not to build something isn't the same as the class being FORBIDDEN from being built

ABSTRACTION:

A parent class that the language itself PREVENTS anyone from building directly – because it only exists to define a shared category for its children.
→ Shape can never be built directly – Java enforces this, it isn't just a polite suggestion

Part 2 — Abstract Methods: A Contract With No Body (11 min)

Open with a story (2 min)

"Imagine a company posts a job description that says: 'Every Sales Representative MUST submit a monthly sales report.' The job description doesn't say HOW to write the report — it just states that submitting one is REQUIRED. Each region writes their report differently, based on their own local format.

*The job description is a **contract**: it demands that the action happens, without dictating how. That is exactly what an abstract method is."*

Definition

An **abstract method** is an action declared in the parent class with NO body — no actual instructions inside it. It exists purely to say: "Every child of mine **MUST** provide their own version of this action." A child that fails to do so is not allowed to exist as a real, buildable object.

Abstract Method vs Regular Method — Side-by-Side

REGULAR METHOD (has a body)

Contains real instructions that actually run

Children MAY override it if they want different behavior – it's optional

Example: `Person.introduce()`
"Hi, I am [name]" – a real, usable default already exists

ABSTRACT METHOD (no body)

Contains NO instructions – just the action's name and what it expects/returns

Children MUST override it – there is no default to fall back on

Example: `Shape.area()`
just says "every Shape MUST have an `area()` – figure out your own formula"

Why Force It? The Safety Net

"What happens if a new `Triangle` class extends `Shape` but the developer **FORGETS** to write an `area()` method?"

WITHOUT AN ABSTRACT METHOD:

`Triangle` silently inherits nothing useful for `area()`.
Calling `triangle.area()` either doesn't exist (error way too late) or worse – silently returns something meaningless like `0`, and nobody notices until a report is wrong.

WITH AN ABSTRACT METHOD:

```
public class Triangle extends Shape { }  
Java immediately reports an error:  
    "Triangle must implement the abstract method area()"
```

The mistake is caught the moment the class is written – not months later when a report shows a wrong number.

Part 3 — Abstract Class vs Regular Parent Class (10 min)

Open with a question (2 min)

"We built `Person` as a parent class back in Week 6, and it worked perfectly fine to create a plain `Person` object. Today we're saying `Shape` should NEVER be built directly. What's the actual difference between these two parents?"

Side-by-Side Comparison

REGULAR PARENT CLASS (e.g. Person)	ABSTRACT CLASS (e.g. Shape)
CAN YOU BUILD IT DIRECTLY? Yes – <code>new Person("Chan", 45)</code> is completely valid and useful	No – Java forbids it. You cannot write <code>new Shape()</code>
DOES IT HAVE TO HAVE ABSTRACT METHODS? No – every method can have a real, working default	Usually yes – at least one action with no real default, forcing children to define their own version
WHEN TO USE THIS DESIGN? The parent represents something REAL that could exist on its own (a generic person exists)	The parent represents only a CATEGORY – no real version of "just a Shape" exists
WHAT CHILDREN GET All of the parent's real, working behavior, which they may optionally override	All of the parent's real behavior (if any) PLUS a REQUIREMENT to implement the abstract parts

The Decision Rule

ASK YOURSELF:

"If I created a plain object of this parent class, with no more specific child type – would that object make real sense in the real world?"

YES → regular parent class (like Person)

NO → abstract class (like Shape, BankAccount, Employee)

Part 4 — Mixing Abstract and Concrete: Partial Blueprints (9 min)

Open with a story (2 min)

"Think about a franchise contract again — but this time, imagine the contract says: 'Every branch MUST use our standard cash register process (this part is fixed and identical everywhere). But every branch MUST design its own local menu (this part is required, but each branch decides how).'"

One document. Two kinds of rules: some parts are already decided for everyone (shared, working behavior), and some parts are deliberately left open, but REQUIRED to be filled in (abstract methods)."

Abstract Classes Can Have BOTH Kinds of Methods

Shape (abstract class) contains:

REGULAR METHOD (shared, working behavior – same for all):

describe() → "This shape is called: " + name
(every shape can describe itself the same way)

ABSTRACT METHOD (no default – every child MUST define it):

area() → no body – Circle, Rectangle, and Triangle
must each provide their own formula

THE KEY INSIGHT:

An abstract class is NOT "all rules, no behavior."

- Some actions are already fully written and shared (just like a regular parent class)
- Some actions are deliberately left as REQUIREMENTS, with no default, because no single answer works for every child

This lets a designer say: "Here's everything you get for free. Here's the ONE thing only you can decide."

Part 5 — Java as Illustration (8 min)

Reminder: Java is just how we write these concepts down. Focus on the concept, not the syntax.

Reading Abstraction as a Concept

```
// ABSTRACT PARENT - cannot be built directly
public abstract class Shape {
    protected String name;

    public Shape(String name) {
        this.name = name;
    }

    // REGULAR METHOD - shared, working behavior for every shape
    public void describe() {
        System.out.println("This shape is called: " + name);
    }

    // ABSTRACT METHOD - no body, every child MUST provide one
    public abstract double area();
}

// CHILD - must implement area() or Java will not allow this class to exist
public class Circle extends Shape {
    private double radius;

    public Circle(double radius) {
        super("Circle");
        this.radius = radius;
    }

    @Override
    public double area() {
        return Math.PI * radius * radius;
    }
}
```

```

    }    }    public class Rectangle extends Shape {    private double width;
private double height;    public Rectangle(double width, double height) {
    super("Rectangle");    this.width = width;    this.height = height;    }
    @Override    public double area() {    return width * height;    }
}

```

What Happens If You Try to Skip It

```

Shape s = new Shape("Generic");
// ERROR: Shape is abstract; cannot be instantiated
// Java refuses this at the moment it is written – not later.

public class Triangle extends Shape {
    // (forgot to write area())
}
// ERROR: Triangle is not abstract and does not override
// the abstract method area() in Shape
// Java catches the missing implementation immediately.

```

Polymorphism Still Works Exactly the Same Way

```

Shape[] shapes = {
    new Circle(5),
    new Rectangle(4, 6)
};

for (Shape s : shapes) {
    s.describe();    // shared behavior – same for all
    System.out.println(s.area()); // abstract behavior – each own formula
}

```

Reading as Plain English

CONCEPT TRANSLATION TABLE

Java code	OOP concept meaning
abstract class Shape	"This is only a category – no one may build a plain Shape object"
public abstract double area()	"Every child MUST provide their own version – no default exists"
describe()	Shared, working behavior – every child gets this for free
new Shape("Generic")	Forbidden – caught immediately as

Test Checkpoint

- ☐ Can explain why some parent classes (like Shape) should never be built directly, using a real-world “category, not a real thing” example
- ☐ Can define “abstract method” as an action with no body that forces every child to implement it
- ☐ Can state the decision rule for choosing abstract class vs regular parent class
- ☐ Can explain why an abstract class can contain BOTH regular methods and abstract methods
- ☐ Can predict the Java error that occurs when a child class forgets to implement an inherited abstract method

Common Misconceptions

Misconception	Clarification
“Abstract class just means a parent class that isn’t used much”	No — “abstract” is an enforced rule. Java physically prevents you from creating an object of an abstract class, no matter how much or little it’s “used.”
“Every parent class should be abstract”	No — only make a class abstract when a plain, standalone object of it would not make real-world sense (Shape). Person from Week 6 is a fine regular parent class because a plain Person genuinely can exist.
“An abstract class can only contain abstract methods”	False — abstract classes often mix regular methods (shared, working behavior) with abstract methods (required, no default). This is one of their biggest strengths.
“If a class has one abstract method, all its methods must also be abstract”	No — you can have any combination. A class can have 5 regular methods and 1 abstract method, as long as at least one method has no body.
“Abstract methods and method overriding are unrelated”	They’re connected — an abstract method REQUIRES overriding (it’s mandatory, not optional), while a regular inherited method only ALLOWS overriding (it’s optional). Both use the same override mechanism from Week 7.

Extension Challenges

● Basic — Should It Be Abstract?

For each parent class below, decide whether it should be a regular parent class or an abstract class. Justify your answer using the decision rule from Part 3.

1. `Animal` — with children `Dog`, `Cat`, `Bird`
2. `Person` — with children `Student`, `Teacher` (from Week 6)
3. `PaymentMethod` — with children `CashPayment`, `CardPayment`
4. `Book` — with children `EBook`, `AudioBook`, `PaperBook`

● Intermediate — Regular or Abstract Method?

For the abstract class `Employee` with children `Manager` and `Intern`, decide whether each method below should be a REGULAR method (shared, working default) or an ABSTRACT method (no default, must be implemented by each child). Explain why.

1. `getEmployeeId()` — every employee gets an ID the exact same way, using a shared counter
2. `calculatePay()` — Managers and Interns are paid using completely different formulas
3. `clockIn()` — every employee clocks in using the exact same shared time-tracking system
4. `getJobDuties()` — every employee type has a completely different list of responsibilities

● Advanced — Design a Mixed Abstract Class

Design (in plain English, no code required) an abstract class `MusicalInstrument` with children `Piano`, `Guitar`, and `Drum`.

- Identify at least ONE action that should be a REGULAR method with shared behavior for all instruments (e.g., something every instrument does the same way).
 - Identify at least ONE action that MUST be an ABSTRACT method because each instrument produces sound completely differently.
 - Explain, in one sentence, why `MusicalInstrument` itself should never be built directly.
-

Lesson Summary

TODAY'S KEY TAKEAWAYS

1. ABSTRACTION (via abstract classes) defines a parent that represents only a CATEGORY – never meant to be built as a real, standalone object.
2. An ABSTRACT CLASS cannot be instantiated directly. Java enforces this rule – it is not just a design suggestion.
3. An ABSTRACT METHOD has no body. It is a CONTRACT that forces every child to provide its own implementation – there is no default to fall back on.
4. Abstract classes can mix REGULAR methods (shared, working behavior) with ABSTRACT methods (required, no default) – giving children free behavior AND required behavior in the same parent.
5. THE DECISION RULE: if a plain object of the parent class would not make real-world sense on its own, make it abstract.
6. Abstraction and Polymorphism (Week 9) work together: abstract methods GUARANTEE that every child has properly overridden the shared action, so polymorphic loops always work correctly, for every current and future child type.

Homework (Prep for Interfaces, Week 11)

Design the following system in plain English — code is optional (bonus only):

Abstract `PaymentMethod` → `CashPayment`, `CardPayment`, `QRPayment` hierarchy:

1. `PaymentMethod` (abstract): attribute: `amount` |
 - REGULAR method: `printReceipt()` — shared behavior: prints “Receipt: \$[amount] paid”
 - ABSTRACT method: `processPayment()` — no default, each child must define its own way of actually processing the payment
2. `CashPayment` extends `PaymentMethod`: overrides `processPayment()` — counts physical bills, calculates change

3. `CardPayment` extends `PaymentMethod` : overrides `processPayment()` — contacts the bank, waits for approval
4. `QRPayment` extends `PaymentMethod` : overrides `processPayment()` — generates a QR code, waits for a scan

For each method, state clearly: is it REGULAR or ABSTRACT, and why? Then write, in plain English, what would happen if you tried to write `new PaymentMethod(50)` directly — and why Java would refuse it.

Concept to carry forward: Notice that `PaymentMethod` still has ONE regular, shared method (`printReceipt()`). Next week we look at a DIFFERENT kind of abstraction — one where a class defines ONLY a pure contract, with NO shared behavior at all: **Interfaces** (Week 11).

Week 10 of 16 | Object-Oriented Programming Concepts | @KhmerSide | syuthd.com